

ML Architecture Guide: The Predictive Procurement Pipeline

This document outlines how our pipeline intercepts BigQuery ML forecasts and translates them into safe, actionable procurement metrics. We will track this process using a single running example in our bookstore database: The Standard Blue Notebook.

Phase 1: The Raw AI Forecast (BigQuery)

ARIMA (AutoRegressive Integrating Moving Average) is a mathematical model that purely looks at the historical past of a dataset to make forecasts. Specifically, it looks at the direct relationship between current and past observations and mathematically flattens the data by looking at the difference between days of sales, rather than raw sales numbers. This is because **ARIMA** doesn't work well with trends or large spikes in purchases. It also looks at the "shocks" or errors from past predictions. For example, if the model under-predicted yesterday, it adjusts upward today to compensate.

For our purposes, we're using BigQuery ML (**ARIMA_PLUS**) to analyze historical sales and we chose **ARIMA_PLUS** because it automatically detects university seasonality (like the Fall/Spring rush). ARIMA_PLUS is an automatic data pipeline that Google built to handle messy, retail data without requiring human intervention.

When we call **ARIMA_PLUS** in BigQuery, it silently runs through a massive pre-processing checklist:

1. **Missing Data Imputation:** If the store system was down on Tuesday and recorded **NULL** sales, BigQuery automatically interpolates the missing value so the math doesn't crash.
2. **Anomaly Scrubbing:** If someone bulk-ordered 500 notebooks for a one-off event, the algorithm flags it as an outlier and smooths it down to the baseline so it doesn't skew next year's forecast.
3. **Holiday Effects:** It automatically adjusts for major calendar events (which can shift dates year-over-year, like Thanksgiving or Easter).
4. **Auto-Tuning:** It runs dozens of parallel ARIMA models with different p,d,q parameters, scores them, and automatically selects the most accurate combination for that specific SKU.

ARIMA_PLUS solves three specific architectural requirements for our procurement system:

1. Zero Data Movement (Operational Safety)

By using BigQuery ML, we don't have to export gigabytes of sensitive sales data to a separate Python microservice or Vertex AI instance. The model trains exactly where the data lives, in the BigQueryML database.

2. Built-in Seasonality Handling

Our datasets are highly cyclical with some variation between each dataset. The model must be able to understand the massive, predictable spikes that occur at the start of a 10-week academic quarter and the dead zones during winter break. **ARIMA_PLUS** automatically detects these multi-layered seasonal patterns (weekly, monthly, and quarterly rhythms) and builds them into the forecast without requiring us to write custom feature engineering.

3. Native Confidence Intervals (The Safety Stock Engine)

Deep learning models often just spit out a single prediction number. **ARIMA_PLUS** inherently calculates probability distributions. It natively outputs the 95% Upper Bound (U_{95}) and Lower Bound (L_{95}).

Every time the model is iterated, the model looks 90 days into the future and outputs three numbers:

- **Predicted Demand (D_p):** The AI's exact best guess for future sales.
- **The Upper/Lower Bounds (U_{95} and L_{95}):** The 95% Confidence Interval. This is the model admitting its own margin of error.

The Blue Notebook Scenario

The model predicts we will sell exactly 500 notebooks this quarter (D_p). However, based on historical variance, it establishes a 95% confidence interval between 400 (L_{95}) and 650 (U_{95}) notebook.

Phase 2: Building the Safety Buffer (FastAPI)

The raw data is sent to our Python middleware. Before we calculate what to buy, we must calculate how much risk we are taking. We use the model's own uncertainty to build our buffers.

- **Dynamic Safety Stock:** Instead of a flat 20% rule, we prepare for the model's worst-case scenario. We subtract the predicted demand from the upper bound to find our emergency buffer. $Safety\ Stock = \max(0, U_{95} - D_p)$
- **Model Certainty Score (C):** We need to tell the UI how much to trust this prediction. If the gap between the upper and lower bound is massive, the model is guessing. We convert this interval width into a 10-99% reliability score.

$$C = \max(10, \min(99, (1 - \frac{U_{95} - L_{95}}{2 \cdot D_p}) \cdot 100))$$

The Blue Notebook Scenario

Our Upper Bound is 650, and our Target is 500. $650 - 500 = 150$ so our Safety Stock is 150 units. Because the interval width (250) is relatively tight compared to the target, our math gives this prediction a strong 75% Certainty Score.

Phase 3: The Procurement Math (FastAPI)

Now that we have our target and our safety buffer, the middleware calculates the exact figures the procurement officer needs to cut a Purchase Order.

- **Lead Time Demand (LTD):** How many units will we sell while waiting for the delivery truck? We calculate the daily run-rate and multiply it by the supplier's lead time.
- **The Reorder Point (ROP):** The system's way to tell the user to reorder the item because the stock is getting low. The exact inventory level that triggers the UI's "Critical Reorder" alert. $ROP = LTD + Safety\ Stock$
- **Suggested Order Quantity (SOQ):** The final, actionable number. How much to buy right now to cover the quarter and the safety buffer, minus what we already own. $SOQ = \max(0, (D_p + Safety\ Stock) - Current\ Stock)$

The Blue Notebook Scenario

We sell ~5.5 notebooks a day. The supplier takes 14 days to deliver, meaning our Lead Time Demand is 78. We add our Safety Stock (150) to get a Reorder Point of 228. If we currently have 200 notebooks in stock, then the UI will suggest an order of 450 units (500 target + 150 buffer - 200 current).

Phase 4: Safety Nets

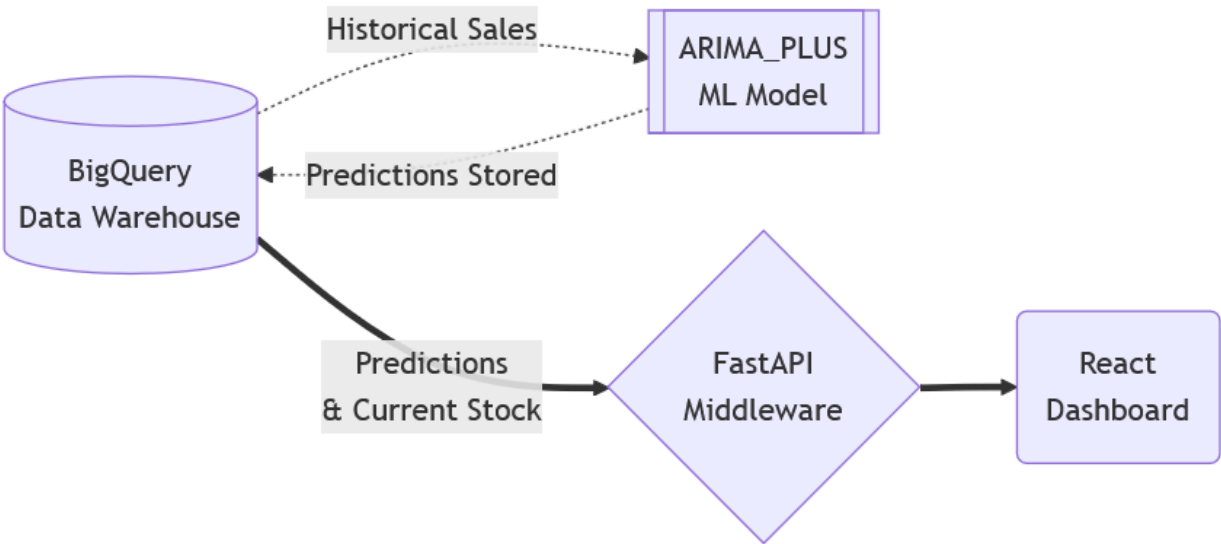
Time-series forecasting on retail data can occasionally produce mathematical anomalies. To ensure the React UI never crashes or suggests a catastrophic order, the middleware enforces two strict rules:

1. **Floor Clamping:** All quantitative outputs (SOQ , ROP) are passed through a $\max(0, \text{value})$ function. We never send negative inventory metrics to the client, preventing broken CSS width calculations on the dashboard.
2. **Lumpy Demand Fallback:** ARIMA struggles with items that sell 0 units for a month, then 50 in one day. If an item has fewer than 5 transactions a month, the middleware bypasses the AI completely. It defaults to a 30-day Simple Moving Average (SMA), setting the `is_using_fallback` boolean to true so the UI knows the AI was bypassed.

Vocabulary & Terms

Metric	Formula / Logic	Purpose
Safety Stock	$\max(0, U_{95} - D_p)$	Buffers against demand spikes.
Certainty Score	Interval Width vs. Target Ratio	UI indicator of AI reliability.
Lead Time Demand	$Daily\ Run\ Rate \cdot Lead\ Days$	Expected sales during transit.
Reorder Point	$LTD + Safety\ Stock$	Triggers the dashboard alert.
Suggested Order	$\max(0, (D_p + Safety) - Stock)$	The final quantity to purchase.

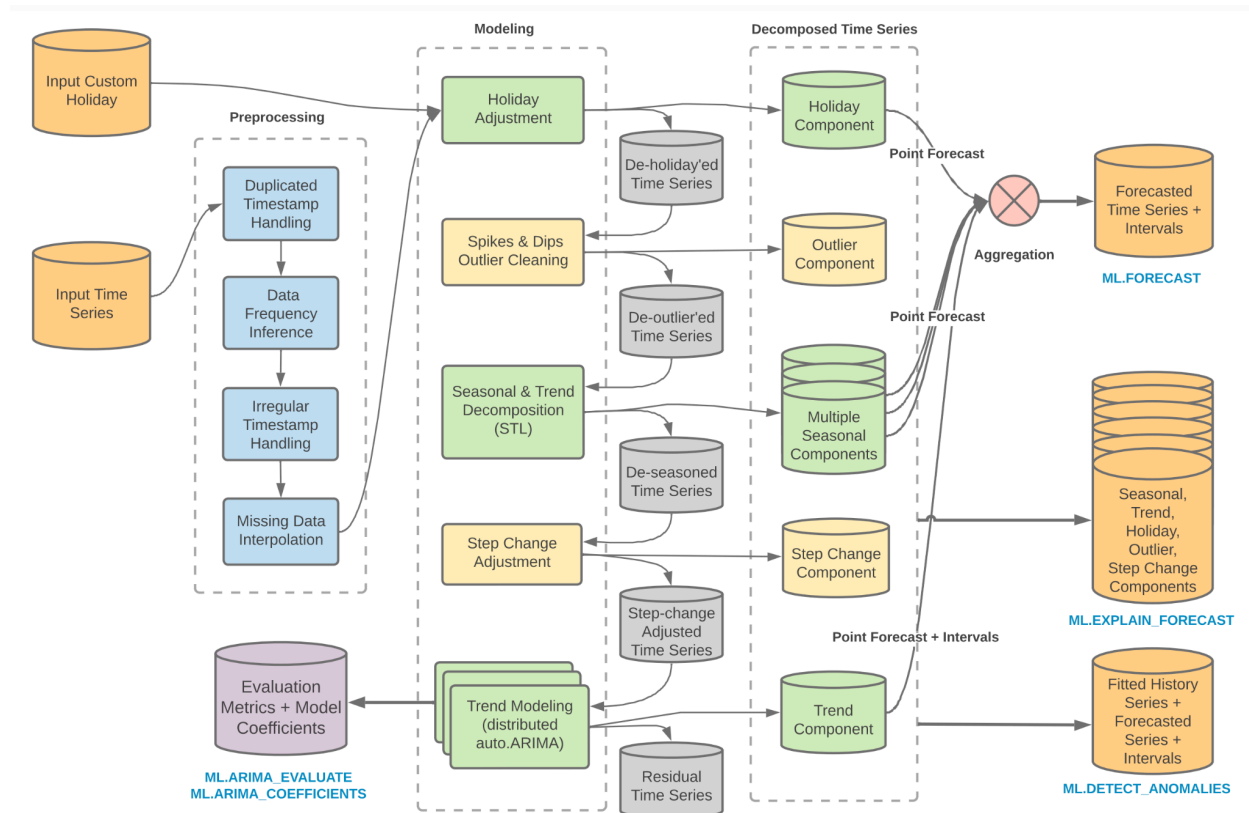
The System Diagram



Resources

1. <https://www.ibm.com/think/topics/arima-model>
2. <https://docs.cloud.google.com/vertex-ai/docs/tabular-data/forecasting-arima/overview>
3. <https://sites.google.com/site/econometricsacademy/econometrics-models/time-series-arima-models>
4. <https://docs.cloud.google.com/bigquery/docs/reference/standard-sql/bigqueryml-syntax-create-time-series>

ARIMA_PLUS Data Pipeline Diagram



Courtesy of [Google](#)

Last updated: 04/30/2026